

Vector `emplace_back` in C++ STL

Last Updated : 26 Nov, 2024



In C++, the **vector `emplace_back()`** is a built-in method used to insert an element at the end of the vector by constructing it **in-place**. It means that the element is created directly in the memory allocated to the vector avoiding unnecessary copies or moves.

Let's take a quick look at a simple example that illustrates vector `emplace_back()` method:

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    vector<int> v;

    // Inserting elements
    v.emplace_back(1);
    v.emplace_back(9);
    v.emplace_back(5);

    for (auto i : v)
        cout << i << " ";
    return 0;
}
```

Output

```
1 9 5
```

This article covers the syntax, usage, and common queries of vector `emplace_back()` method in C++:

Table of Content

- [Syntax of Vector `emplace\_back\(\)`](#)
- [Examples of vector `emplace\_back\(\)`](#)

- [Inserting Elements in Vector of Strings](#)
- [Inserting Elements in a Vector of Custom Type](#)
- [Vector `emplace\_back\(\)` vs `push\_back\(\)`](#)

Syntax of Vector `emplace_back()`

The vector `emplace_back()` is the member method of `std::vector` class defined inside `<vector>` header file.

```
v.emplace_back(val);
```

Parameters:

- **val:** Value to be added. It is forwarded to the constructor of the type of vector.

Return Value:

- Until C++ 17, this function didn't used to return any value, but now, it returns the reference to the inserted value.

Examples of vector `emplace_back()`

The following examples demonstrates the use of vector `emplace_back()` function for different scenarios:

Inserting Elements in Vector of Strings

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    vector<string> v = {"Hi", "Geeks!"};

    // Inserting element at the end
    v.emplace_back("Welcome");

    for (auto i : v)
        cout << i << " ";
    return 0;
}
```

```
}
```

Output

```
Hi Geeks! Welcome
```

Inserting Elements in a Vector of Custom Type

```
#include <bits/stdc++.h>
using namespace std;

// Class that notifies when copied
class A {
public:
    int a;
    A(int x = 0) {
        a = x;
    }
    A(const A& other) {
        a = other.a;
        cout << a << "'s CC called\n";
    }
};

int main() {
    vector<A> v;

    v.reserve(5);

    // Inserting element using emplace_back()
    v.emplace_back(1);
    v.emplace_back(9);

    // Inserting element using push_back()
    v.push_back(5);

    return 0;
}
```

Output

5's CC called

From the above example, we can confirm that `vector::emplace_back()` doesn't create extra copies but `vector::push_back()` does.

Vector `emplace_back()` vs `push_back()`

Both functions are used to insert elements at the back of vector but they **differs in the way of insertion**. Following table lists the primary differences of the **vector `emplace_back()`** and **`vector::push_back()`** in C++:

Vector <code>emplace_back()</code>	Vector <code>push_back()</code>
Construct the element in-place directly in the vector.	Create an object to be copied first and then pass it this function.
It is more efficient as it avoids the unnecessary copying or moving	It is less efficient due to copying or moving the object.
Syntax: <code>v.emplace_back(val);</code>	Syntax: <code>v.push_back(val);</code>

 Comment

More info 

Advertise with us

Next Article 

Introduction to C++ Programming
Language

Similar Reads

C++ Programming Language

C++ is a computer programming language developed by Bjarne Stroustrup as an extension of the C language. It is known for its fast speed, low level memory management and is often taught as first...

🕒 5 min read

C++ Overview



C++ Basics



C++ Variables and Constants



C++ Data Types and Literals



C++ Operators



C++ Input/Output



C++ Control Statements



C++ Functions



C++ Pointers and References



Do Not Sell or Share My Personal Information